salesianos
COLEGIO SAN IGNACIO
CÁDIZ

Bases de Datos

UD2: Creación de Bases de Datos relacionales

Félix Reyes Fernández

Contenido

CRITERIOS DE EVALUACIÓN	PROCEDIMIENTOS ESENCIALES	EVALUACIÓN	CONTENIDOS
a) Se ha analizado el formato de almacenamiento de la información. b) Se han creado las tablas y las relaciones entre ellas. c) Se han seleccionado los tipos de datos adecuados. d) Se han definido los campos clave en las tablas. e) Se han implantado las restricciones reflejadas en el diseño lógico. h) Se han utilizando asistentes, herramientas gráficas y los lenguajes de definición y control de datos.	PE 1. Implantar un modelo relacional, definiendo las distintas tablas, relaciones entre tablas, restricciones y tipos de datos.	Prueba específica 2 (70%)	Modelo de datos. Concepto y tipos. Proceso de diseño de una base de datos. Terminología del modelo relacional. Relaciones, atributos, tuplas. Representación. Diagramas relacionales. Lenguaje de definición de datos. Estándar SQL. La sentencia CREATE. Concepto de objeto de una base de datos. Tipos de datos. Juegos de caracteres. Criterios de comparación y ordenación. Claves primarias. Claves primarias simples y compuestas. Índices. Características. Valores no duplicados. El valor NULL. Claves ajenas. Opciones de borrado y modificación Implementación de restricciones. Lenguaje de descripción de datos (DDL). Herramientas proporcionadas por sistema gestor para la descripción de datos. Lenguaje de control de datos (DCL). Herramientas gráficas proporcionadas por el sistema gestor para la definición de usuarios, roles y privilegios.
f) Se han creado vistas. g) Se han creado los usuarios y se les han asignado privilegios.	PE 2. Implantar un esquema de seguridad en nuestro SGBD, definiendo usuarios, permisos de usuarios, y distintas vistas a las cuales acceden los usuarios	Prueba específica 3 (30%)	Vistas Usuarios. Roles. Privilegios. Objetos.

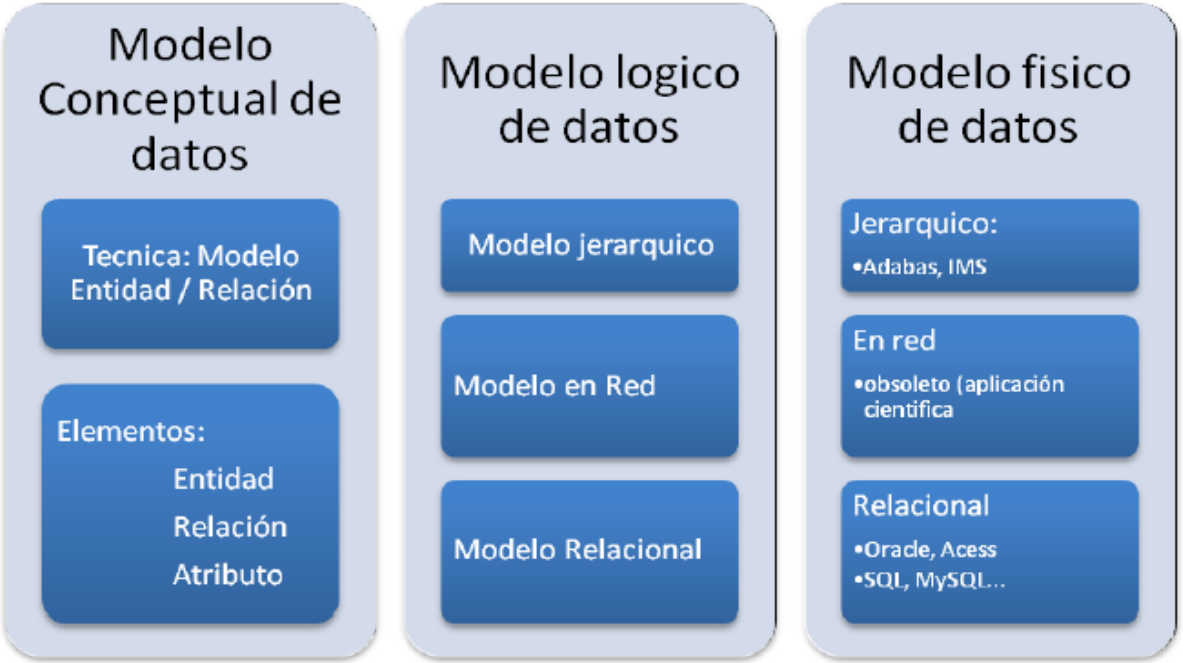
R.A.2. Crea bases de datos definiendo
su estructura y las características de sus elementos según el modelo relacional

Introducción

- **Diseño de una Base de Datos**: Proceso por el que se determina la organización de una base de datos, incluidos su estructura y contenido. Se realiza en tres fases:
  - ***Diseño conceptual:*** representación abstracta y de alto nivel de la realidad.
  - ***Diseño lógico:*** convierte esta representación en especificaciones que pueden implantarse en un ordenador y ser procesadas por él.
  - ***Diseño físico:*** determina las estructuras de almacenamiento físico y métodos de consulta requeridos para un acceso óptimo a la base de datos

Modelos de Datos

- ***Modelos de Datos*** son un conjunto de herramientas que permiten diseñar la base de datos a partir de la observación del mundo real que se quiere representar

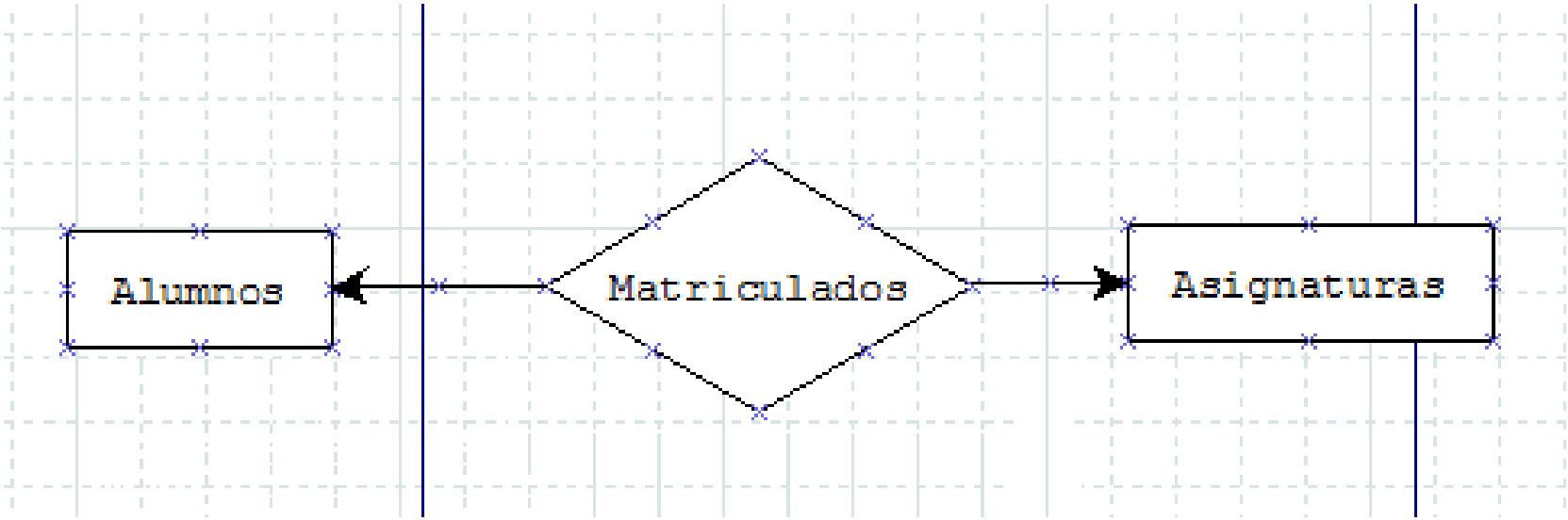


Modelo Conceptual de Datos

- Se utilizan para realizar el *Diseño conceptual.* Ej: Modelo E/R, Modelo Orientado a Objetos, Modelo Binario y Modelo Semántico de Datos

- Modelo Entidad/Relación
  - Presentan la realidad de forma fácil
  - Es el más sencillo de entender
  - Es el más difundido
  - Y por ser bastante adecuado para hacer un diseño de una BD

Modelo Conceptual de Datos



Modelo Lógico de Datos

- Describen datos que ya pueden ser implementados en nuestro sistema. Se basan en registros (normalmente con campos de longitud fija).
- **Modelo relacional**: Se representa por tablas

Modelo Lógico de Datos

- Alumnos(DNI, Nombre, Apellidos, Edad)
- Asignaturas(Codigo, Nombre, Horas)
- Matriculados(DNI_Alumno, Codigo_Asignatura)

Modelo Físico de Datos

- Su misión es representar como se almacenan en estructuras los datos, dedicándose a implementar un SGBD a bajo nivel.

Modelo Físico de Datos

```
SQL> CREATE TABLE Alumnos(
  2  DNI VARCHAR2(9),
  3  Nombre VARCHAR2(20),
  4  Apellidos VARCHAR2(20),
  5  Edad NUMBER);

Table created.
```

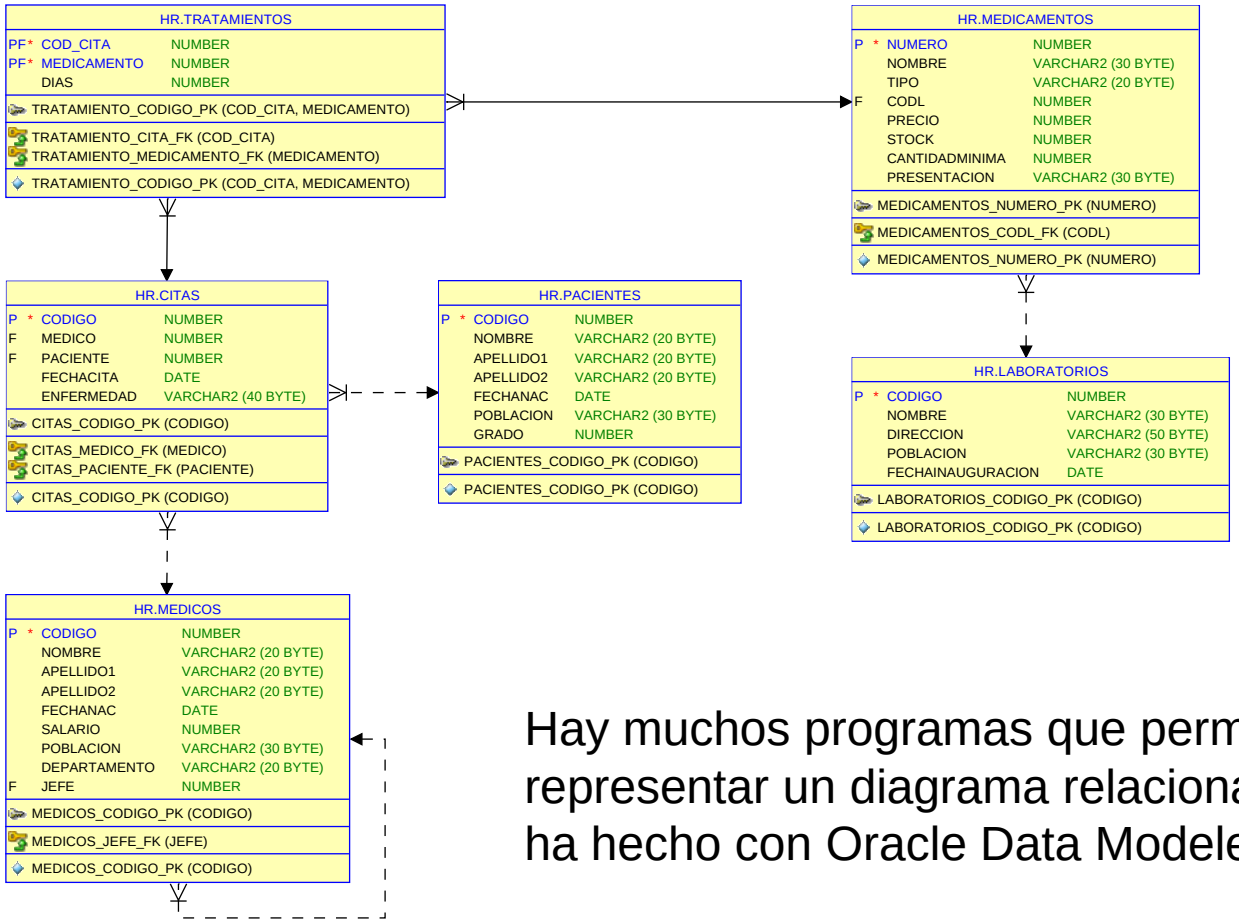
Terminología del modelo relacional

- **Modelo Relacional:** Herramienta para realizar Diseño Lógico
- **Relación:** Elemento básico del modelo relacional
  - Nombre de tabla
  - Conjunto de columnas (atributos)
  - Conjunto de filas (tuplas)
- **Tupla:** fila de la tabla que se corresponde con cada ocurrencia del elemento de la tabla.
- **Atributo:** columna de la tabla. Mismo significado que en E/R.
- **Cardinalidad:** Número de filas de la tabla.
- **Grado:** Número de atributos de la tabla.

Conceptos básicos

Código	Nombre	Apellido1	Apellido2	Sexo	Edad
1	Juan	Pérez	Sánchez	Hombre	22
2	Sonia	Delgado	Cuenca	Mujer	23
3	Jose	Reyes	Aleu	Hombre	21
4	Francisco	Espadas	Reus	Hombre	27

Representación. Diagramas relacionales



Hay muchos programas que permiten representar un diagrama relacional. Este se ha hecho con Oracle Data Modeler

Conceptos básicos

- **Clave**: Conjunto de atributos que identifican de manera única a cada tupla de la tabla
  - Candidatas, Primarias, Alternativas o Secundarias
  - Foráneas: Conjunto de atributos que es clave en otra tabla

TABLA DEPARTAMENTOS

CLAVE PRIMARIA CLAVE CANDIDATA

Nº DEPARTAMENTO	NOMBRE DEPARTAMENTO	PRESUPUESTO
D1	MARQUETING	1000
D2	DESARROLLO	1200
D3	INVESTIGACIÓN	5000

TABLA EMPLEADOS

CLAVE PRIMARIA CLAVE AJENA

Nº EMPLEADO	APELLIDOS	Nº DEPARTAMENTO	SALARIO
E1	LOPEZ	D1	500
E2	FERNANDEZ	D3	1000
E3	GARCÍA	D2	1200

Clave

- Se denomina **clave** a un grupo de atributos que determinan unívocamente todas las ocurrencias de una entidad
- Clave Primaria, Alternativa o Secundaria
- Clave Simple-Clave Compuesta
- **Clave foránea**

Reglas de Integridad

- Son restricciones a la hora de diseñar una base de datos que ayudan a mantener coherencia y veracidad de la información en la Base de Datos.
- **Inherentes al modelo (Características propias de una relación)**
  - No hay dos tuplas iguales,
  - El orden de la tuplas y los atributos no es relevante,
  - Cada atributo sólo puede tomar un único valor del dominio al que pertenece y,
  - Ningún atributo que forme parte de la clave primaria de una relación puede tomar un valor nulo.
- **Semánticas o de usuario**
  - Restricción de clave primaria (PRIMARY KEY)
  - Restricción de unicidad (UNIQUE, Claves Candidatas)
  - Restricción de obligatoriedad (NOT NULL)
  - Restricción check o de verificación
  - Restricción de clave ajena o integridad referencial

Reglas de Integridad

- **Integridad Referencial**: Generalmente, lo ideal es evitar claves foráneas que puedan contener valores nulos, pero no siempre ocurre esto.
- **La regla de integridad referencial** nos indica que todos los valores que tenga una clave foránea en una tabla, tienen que coincidir con los valores de la clave primaria a la que apuntan. (En determinados sistemas, podemos ampliar esto diciendo que pueden ser nulas)

- Borrado y/o modificación en cascada (**CASCADE**)

- Borrado y/o modificación restringido (**RESTRICT**): No se puede borrar ni modificar si existen tuplas hijas.

- Borrado y/o modificación con nulos (**SET NULL**)

- Borrado y/o modificación con valor por defecto (**SET DEFAULT**)

Lenguaje de definición de datos.

- DDL (Data Definition Language) → Permite a los usuarios de la misma llevar a cabo las tareas de definición de las estructuras que almacenarán los datos).

- La definición de la estructura de la base de datos incluye tanto la creación inicial de los diferentes objetos que formarán la base de datos, como el mantenimiento de esa estructura.

- Las principales funcionalidades del DDL son la creación, modificación y borrado de las tablas que componen la base de datos, así como de los índices, vistas, sinónimos, permisos, etc. que pudieran definirse sobre las mismas.

SQL*PLUS

- SQL*Plus es un entorno en el cual puedes hacer lo siguiente:
  - Ejecutar sentencias SQL para recuperar, modificar, añadir y eliminar datos de la base de datos.
  - Formatear, realizar operaciones de cálculo, almacenar e imprimir resultados de consultas en forma de informes.
  - Crear ficheros de scripts para almacenar sentencias SQL para usos repetitivos en el futuro.

Comandos SQL*Plus

- **STA[RT]** filename [.ext]: Arranca un fichero de comandos almacenado previamente
- **@filename:** Igual que START
- **EXIT:** Abandonar SQL*Plus
- **CLEAR SCR**: Limpiar la pantalla
- **SPOOL y SPOOL OFF**: Sirve para almacenar todo lo que hacemos en ficheros
- **DESCRIBE:** Lista los atributos de las tablas
- **SET PAUSE ON:** Hace que SQL*Plus pare al principio de cada página. Sólo se pasa a la siguiente página cuando el usuario pulsa RETURN.
- **SET LINESIZE TAMAÑO:** establece la longitud de las líneas en caracteres
- **SHOW LINESIZE:** muestra el tamaño actual de las líneas
- **SET PAGESIZE TAMAÑO:** establece el número de líneas por página
- **SHOW PAGESIZE:** muestra el número de líneas por página
- **COLUMN columna FORMAT a50**: Darle más longitud a una columna (no numérica)
- **SHOW USER:** Muestra el usuario que está conectado.

Tipos de Objetos de una Base de Datos

Objeto	Descripción
Tabla	Unidad básica de almacenamiento, formada por filas y columnas
Vista	Representación lógica de un subconjunto de datos extraídos de una o más tablas
Secuencia	Estructura que genera valores automáticos
Índices	Variables que mejoran la eficiencia de consultas a las bases de datos
Sinónimos	Estructuras que dan nombres alternativos a otros objetos

Un objeto es un tipo particular de recurso que nos sirve para realizar acciones dentro de nuestra base de datos

Tipos de Objetos de una Base de Datos

- Los nombres de las tablas y de las columnas deben empezar con una letra y no deben superar los 30 caracteres de longitud
- Deben contener solo los caracteres A-Z, a-z, 0-9, _, $, #
- Los nombres no deben duplicarse para el mismo usuario, pero si pueden ser igual para distintos (Concepto de esquema)
- No deben ser palabras reservadas de ORACLE.
- Deben usarse nombres descriptivos para cualquier objeto de la base de datos
- En el caso de tener campos relacionados, se debe intentar poner el mismo nombre para los dos.
- No es sensible a las mayúsculas y minúsculas

Tipos de Datos de una Base de Datos

Tipo de dato	Descripción
VARCHAR(n)	Almacena datos alfanuméricos de longitud variable. Si el valor insertado es menor que N, el resto lo ajusta al tamaño del valor → Ganamos espacio de almacenamiento
CHAR(n)	Almacena datos alfanuméricos de longitud fija. Si el valor insertado es menor que N, el resto lo rellena con espacios en blanco.
NCHAR y NVARCHAR	Almacena datos alfanuméricos de longitud fija-variable usando caracteres Unicode.

Tipos de Datos de una Base de Datos

Tipo de dato	Descripción
NUMBER(p, s)	Almacena números. La p representa los dígitos totales, y la s los dígitos decimales.
DATE	Almacena fechas y horas concretas.
TIMESTAMP	Más específico que Date, almacenando incluso fracciones de segundo.

Manipulación de tablas

- **Creación de tablas**

```
CREATE TABLE nombre_tabla (
    nombre_columna1 tipo_dato [DEFAULT expresión],
    nombre_columna2 tipo_dato [DEFAULT expresión],
    .....
)
```

Manipulación de tablas

- **Creación de tablas: Ejercicio**

- Crear una tabla que sirva para almacenar los datos personales de una serie de personas. Concretamente el dni, nombre, apellidos, fecha nacimiento y número de hermanos.

Manipulación de tablas

- **Creación de tablas: Ejercicio 1**

```
CREATE TABLE persona (
        dni CHAR(9),
        nombre VARCHAR(50),
        apellidos VARCHAR(50),
        fecha_nac DATE,
        numero_hermanos NUMBER(2));
```

- **Describe (desc) Nombre_tabla**

Manipulación de tablas

- **Creación de tablas: Ejercicio (opción DEFAULT)**

- Crear una tabla que sirva para almacenar los datos personales de una serie de personas. Concretamente el dni, nombre, apellidos, fecha de alta en el sistema y número de hermanos). Especifica que la fecha de alta en el sistema sea por defecto la fecha actual del sistema.

Manipulación de tablas

- **Creación de tablas: Ejercicio 2**

```
CREATE TABLE persona
        (dni CHAR(9),
        nombre VARCHAR(50),
        apellidos VARCHAR(50),
        fecha_alta DATE DEFAULT SYSDATE,
        numero_hermanos NUMBER(2));
```

Manipulación de tablas

- **Modificar Estructura Tablas: Alter Table**

```
ALTER TABLE  nombre_tabla
ADD (nombre_columna tipo_dato [DEFAULT
expresion],
            nombre_columna tipo_dato [DEFAULT
expresion],...)
```

```
ALTER TABLE  nombre_tabla
MODIFY (nombre_columna tipo_dato [DEFAULT
expresion],
            nombre_columna tipo_dato [DEFAULT
expresion],...)
```

Manipulación de tablas

- **Modificar Estructura Tablas: Alter Table**

```
ALTER TABLE  nombre_tabla
DROP COLUMN nombre_columna
```

Manipulación de tablas

- **Modificar Estructura Tablas: Ejercicio**
- Modifica el sistema para que pueda añadir datos acerca del número de hijos que tuviese cada persona y que por defecto valga 0
- Modifica el sistema para que se puedan almacenar en el campo apellidos 35 bytes mas de los que tiene.
- Debido a restricciones del problema, el sistema no tiene porque guardar información acerca del numero de hermanos de cada persona. Realiza los cambios necesarios en el sistema.

Manipulación de tablas

- **Modificar Estructura Tablas: Ejercicio 3**

```
ALTER TABLE  persona
ADD (numero_hijos NUMBER DEFAULT 0);
```

```
ALTER TABLE  persona
MODIFY (apellidos VARCHAR(85));
```

```
ALTER TABLE  persona
DROP COLUMN numero_hermanos;
```

Manipulación de tablas

- **Cambiar Nombre Objeto: Rename**

```
RENAME nombre_antiguo TO nombre_nuevo;
```

- **Ejercicio 4:** Cambiar el nombre de la tabla persona por el de socios

```
RENAME persona TO socios;
```

Manipulación de tablas

- **Truncate Table:** borra todas las filas de una tabla. Vacía su contenido y libera el espacio de memoria ocupado por esos datos, dejando intacta la estructura de la tabla.

```
TRUNCATE TABLE nombre_tabla;
```

- **Ejercicio 5**: Dar de baja a todas las personas de la tabla persona

```
TRUNCATE TABLE persona;
```

Manipulación de tablas

- **Drop Table:** libera el espacio de memoria ocupado tanto por las filas como por las columnas. Borra los datos de una tabla y la tabla en sí.

```
DROP TABLE nombre_tabla;
```

- **Ejercicio 6:** Dar de baja a la tabla persona

```
DROP TABLE persona;
```

Concepto de CONSTRAINT

- Mecanismos que hacen cumplir una serie de reglas a la hora de insertar valores dentro de los campos de una tabla. Previenen acerca de datos no válidos en las tablas.
- Se pueden usar para:
  - Prevenir el borrado de una tabla si existen dependencias con otras tablas de la base de datos
  - Hace cumplir reglas a nivel de tabla ya sea insertando, modificando o borrando datos.

Concepto de CONSTRAINT

Restricción	Descripción
NOT NULL	La columna no puede tener valores nulos
UNIQUE	La/s columna/s deben tener valores únicos
PRIMARY KEY	Definimos la clave primaria de una tabla
FOREIGN KEY	Relaciona datos de diferentes tablas mediante integridad referencial
CHECK	Una determina condición que debe cumplirse para una columna

Almacenadas en USER_CONSTRAINTS

Formato de definición de CONSTRAINT

- **Nombre restricción**

nombretabla_nombrecolumna_abreviaturarestricción

- **Si no se especifica un nombre, Oracle crea uno automáticamente**
- **Definiciones a nivel de columna**

```
CREATE TABLE nombre_tabla
(columna tipo_dato [DEFAULT expresión],
...,columna tipo_dato [CONSTRAINT nombre_restricción] tipo_restricción,...)
```

- **Definiciones a nivel de tabla**

```
CREATE TABLE nombre_tabla
(columna tipo_dato [DEFAULT expresión],
..., CONSTRAINT nombre_restricción tipo_restricción (columna), ...)
```

Formato de definición de CONSTRAINT

- **Visualizando restricciones en el Diccionario de Datos**

```
SELECT constraint_name, constraint_type
FROM user_constraints
WHERE table_name='nombre_tabla';
```

```
SELECT constraint_name, column_name
FROM user_cons_columns
WHERE table_name='nombre_tabla';
```

Tipos de Constraints. Chequeando Constraints

- **NOT NULL:** Solo a nivel de columna

```
CREATE TABLE nombre_tabla
(columna tipo_dato NOT NULL,
...)
```

- **Ejercicio:** Se desea crear una tabla para almacenar los datos de los alumnos de primero de DAM. Concretamente se desea guardar el dni, nombre, apellidos y el teléfono móvil de todos los alumnos, siendo obligatorio que nos lo faciliten.

Tipos de Constraints. Chequeando Constraints

- **Ejercicio 7 NOT NULL**

```
CREATE TABLE alumnos_NN
        (dni CHAR(9),
        nombre VARCHAR(50),
        apellidos VARCHAR(50),
        teléfono CHAR(9) NOT NULL)
```

Tipos de Constraints. Chequeando Constraints

- **PRIMARY KEY**: Incluye las restricciones UNIQUE y NOT NULL

```
CREATE TABLE nombre_tabla
(columna tipo_dato,
CONSTRAINT nombretabla_nombre_columa_pk
PRIMARY KEY (columna1[, columna2,...]),
```

- **Ejercicio**: Se desea crear una tabla para almacenar los datos de una zapatería. Concretamente se desea almacenar el número identificativo de cada zapato, modelo, color y marca. Define el campo identificativo de cada zapato como clave primaria de la tabla.

Tipos de Constraints. Chequeando Constraints

- **Ejercicio 8 PRIMARY KEY**

```
CREATE TABLE zapateria_PK
            (id_zapato VARCHAR(9),
            marca VARCHAR(50),
            modelo VARCHAR(50),
            color VARCHAR(9) ,
CONSTRAINT zapateria_id_pk PRIMARY KEY
(id_zapato));
```

Tipos de Constraints. Chequeando Constraints

- **UNIQUE**

```
CREATE TABLE nombre_tabla
(columna tipo_dato,
CONSTRAINT nombretabla_nombre_columa_u
UNIQUE (columna), ...)
```

- **Ejercicio:** Se desea crear una tabla para almacenar los datos de una zapatería. Concretamente se desea almacenar el número identificativo de cada zapato, modelo, color y marca. Nuestra zapatería tiene un convenio con los fabricantes y solo se trae el mejor zapato por cada marca.

Tipos de Constraints. Chequeando Constraints

- **Ejercicio 9 UNIQUE**

```
CREATE TABLE zapateria_U
            (id_zapato VARCHAR(9),
            marca VARCHAR(50),
            modelo VARCHAR(50),
            color VARCHAR(9) ,
CONSTRAINT zapateria_marca_u
UNIQUE(marca));
```

Tipos de Constraints. Chequeando Constraints

- **FOREIGN KEY (Borrado en Cascada)**

```
CREATE TABLE nombre_tabla
(columna tipo_dato,
CONSTRAINT tabla_ columa_fk FOREIGN KEY
(columna[,columna2…])
        REFERENCES
tabla_padre(columna[,columna2…])
        ON DELETE CASCADE, ...)
```

- **ON DELETE CASCADE**
- **ON DELETE SET NULL**
- **ON DELETE NO ACTION:** Por defecto (No se especifica la claúsula). Si se intenta eliminar un registro de la tabla referenciada por una "foreign key", Oracle no lo permita y muestre un mensaje de error

Tipos de Constraints. Chequeando Constraints

- **FOREIGN KEY**

- **Ejercicio**: Se desea crear almacenar los datos de las motocicletas que hay en Cádiz y sus propietarios. Concretamente de las motocicletas se desea guardar la matricula, marca, modelo, color y propietario. Sobre los propietarios, se desea almacenar su dni, nombre, apellidos y teléfono. Además, para guardar la consistencia entre las tablas, cuando borremos un propietario se deberá borrar todos los registros relacionados en la tabla motocicletas.

Tipos de Constraints. Chequeando Constraints

- **Ejercicio FOREIGN KEY**

```
CREATE TABLE propietarios
        (dni_propietarios CHAR(9),
        nombre VARCHAR(50),
        apellidos VARCHAR(50),
        telefono CHAR(9),
CONSTRAINT propietario_dni_pk
PRIMARY KEY (dni_propietarios))
```

Tipos de Constraints. Chequeando Constraints

- **Ejercicio FOREIGN KEY**

```
CREATE TABLE motos
            (matricula VARCHAR(9),
            marca VARCHAR(50),
            modelo VARCHAR(50),
            color VARCHAR(9),
            dni_propietario CHAR(9),
CONSTRAINT motos_mat_pk PRIMARY KEY
(matricula),
CONSTRAINT motos_dni_fk FOREIGN KEY
(dni_propietario)
REFERENCES propietarios(dni_propietarios)
                    ON DELETE CASCADE);
```

Tipos de Constraints. Chequeando Constraints

- CHECK

```
CREATE TABLE nombre_tabla
(columna tipo_dato,
CONSTRAINT nombretabla_nombrecolumna_ck
CHECK expresión,...)
```

- **Ejercicio**: Se desea crear una tabla para almacenar los datos de una zapatería. Concretamente se desea almacenar el número identificativo de cada zapato, modelo, color y marca. Define el campo identificativo de cada zapato como clave primaria de la tabla y los zapatos solo pueden ser negros, verdes, azules o dorados.

Tipos de Constraints. Chequeando Constraints

- **Ejercicio CHECK**

```
CREATE TABLE zapateria_CK
            (id_zapato VARCHAR(9),
            marca VARCHAR(50),
            modelo VARCHAR(50),
            color VARCHAR(9) ,
CONSTRAINT zapateria_id_pk PRIMARY KEY
(id_zapato),
CONSTRAINT zapateria_color_ck CHECK (color
='negro'
OR color ='verde'
OR color ='azul'
OR color ='dorado'));
```

Manipulación de restricciones

- **Añadir restricción**
- **A nivel de tabla**

```
ALTER TABLE nombre_tabla
ADD CONSTRAINT nombre_restriccion
TIPO_RESTRICCIÓN (columna);
```

- **A nivel de columna**

```
ALTER TABLE nombre_tabla
MODIFY (columna tipo_dato NOT NULL);
```

Manipulación de restricciones

- **Borrar restricción**

```
ALTER TABLE nombre_tabla
DROP CONSTRAINT nombre_restriccion ;
```

- **Deshabilitar restricción**

```
ALTER TABLE nombre_tabla
DISABLE CONSTRAINT nombre_restriccion ;
```

- **Habilitar restricción**

```
ALTER TABLE nombre_tabla
ENABLE CONSTRAINT nombre_restriccion ;
```

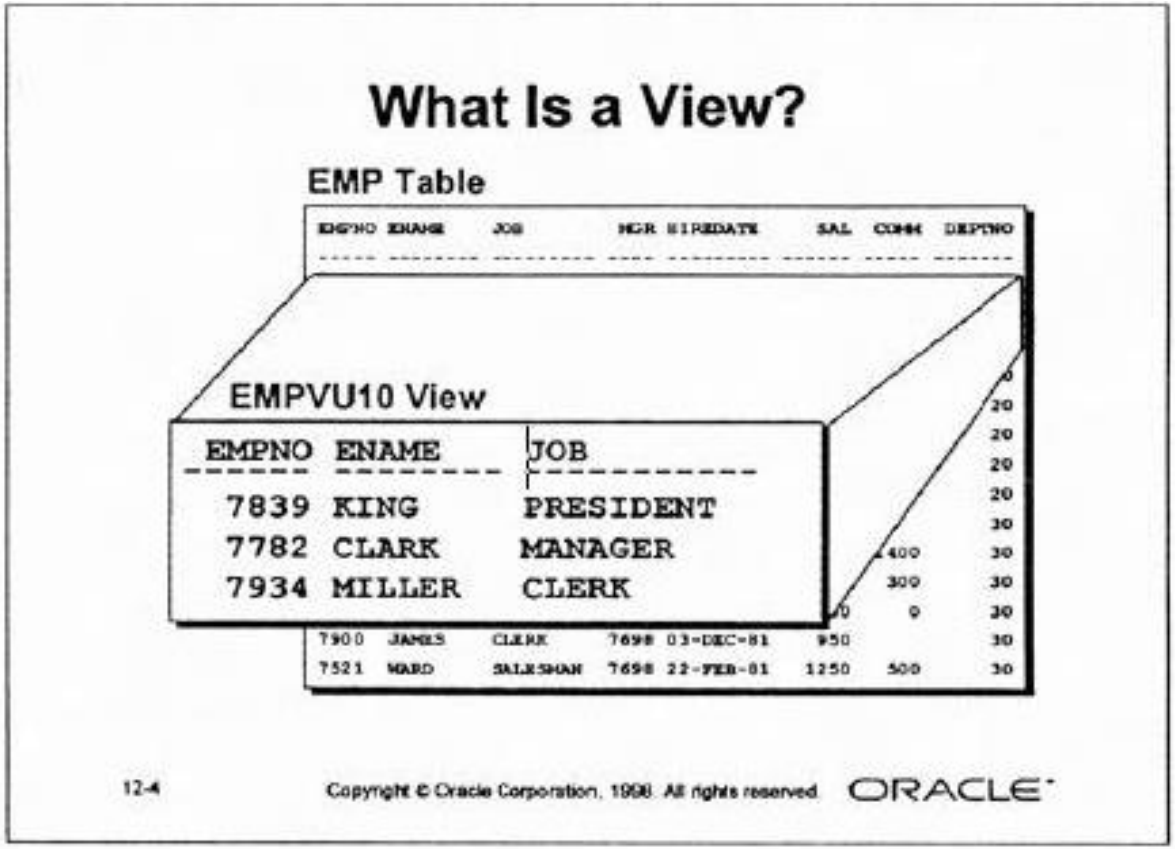
Ejercicios

- 1.- Crea la tabla *departamento* para que almacene la identificación del departamento y el nombre de cada departamento que constituyen una empresa.
- 2.- Crea la tabla *empleado* para que contenga información sobre el identificador de empleado, el nombre, los apellidos y el número de departamento en el que trabaja en la empresa.
- 3.- Modifica la tabla *empleado* para que pueda almacenar nombres más largos.
- 4.- Confirma que ambas tablas, tanto *departamento* como *empleado*, están almacenadas en el diccionario de datos.
- 5.- Crea la tabla empleados2 basada en la tabla empleado, pero que solo contenga el número del empleado, en nombre del empleado y el departamento en el que trabaja.
- 6.- Borra la tabla empleado.
- 7.- Renombra la tabla empleados2 a empleado.

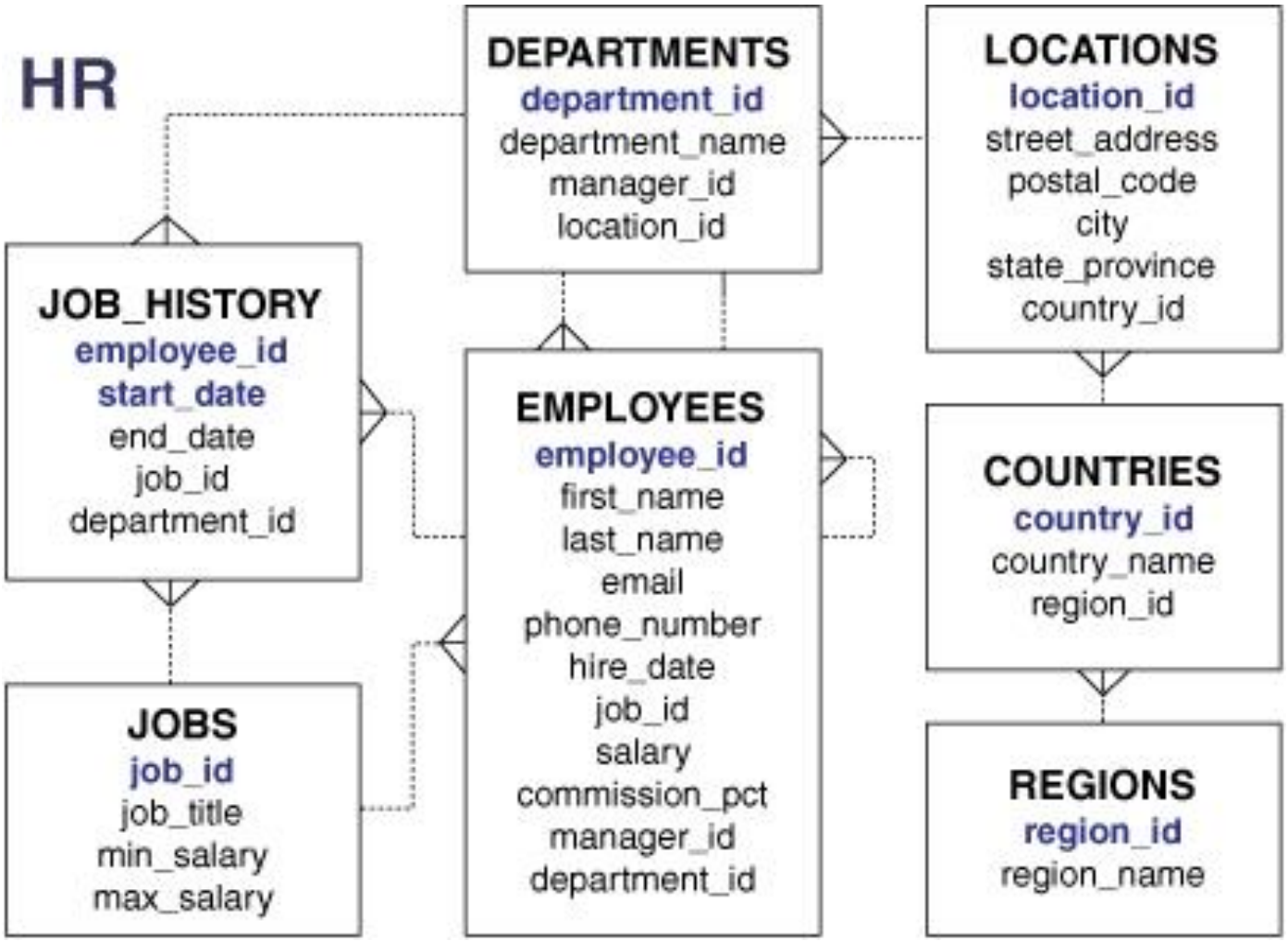
Ejercicios

- **Restricciones (Eliminar primero las restricciones creadas)**
- 1.- Añade a la tabla empleado la restricción PRIMARY KEY usando la columna de ID. La restricción debe especificarse en la creación de la tabla.
- 2.- Crea una restricción PRIMARY KEY en la tabla departamento usando el campo ID. La restricción debe especificarse en la creación de la tabla.
- 3.- Añade una clave foránea a la tabla empleado que asegure que los empleados no se asignan a departamentos inexistentes.
- 4.- Confirma la creación de dichas restricciones consultando el diccionario de datos.

Vistas


What Is a View?
EMP Table
EMPVU10 View
EMPNO ENAME JOB
7839 KING PRESIDENT
7782 CLARK MANAGER
7934 MILLER CLERK
12-4
Copyright © Oracle Corporation, 1998. All rights reserved.
ORACLE

Vistas (BD de ejemplo “HR”)



Vistas

- Tabla virtual que obtiene sus datos de otra/s tabla/s de la base de datos.
- Principales usos desde el punto de vista de la seguridad:
  - Mostrar solo determinados atributos (Por ejemplo: No permitir el acceso a los sueldos)

```sql
CREATE VIEW v_emp AS
    SELECT nombre, puesto
    FROM empleados;
```

  - Mostrar solo determinadas tuplas

```sql
CREATE VIEW v_emp AS
    SELECT nombre, puesto
    FROM empleados;
```

Creación de las vistas

- Sintaxis

```
CREATE VIEW nombre vista
AS consulta select
[WITH CHECK OPTION [CONSTRAINT
nombre restricción]]
[WITH READ ONLY];
```

- **[WITH CHECK OPTION** [CONSTRAINT nombre restricción]]: especifica que solo las filas que cumplen los criterios de la vista pueden insertarse o actualizarse

- **[WITH READ ONLY]** asegura que no puedan ejecutarse sentencias DML en la vista.

Creación de vistas

- **Ejercicio:** Crea una vista, EMPVU30, que contiene detalles de los empleados del departamento 30

Creación de vistas

- **Ejercicio:** Crea una vista, EMPVU30, que contiene detalles de los empleados del departamento 30

```
CREATE [OR REPLACE] VIEW EMPVU30
AS SELECT employee_id, first_name
FROM employees
WHERE department_id=30;
```

Creación de vistas

- La vista se almacena en el Diccionario de Datos (**USER_VIEWS**)
- La subconsulta puede contener complejas sentencias SELECT
- Si no especificas un nombre de restricción para la vista creada con la opción CHECK OPTION, el sistema asignará un nombre por defecto con el formato SYS_Cn

Consultando una vista

- **Usando WITH CHECK OPTION**

```
CREATE  VIEW empvu30
AS SELECT *
FROM employees
WHERE department_id=30
WITH CHECK OPTION;
```

```
UPDATE empvu30
SET department_id=10
WHERE employee_id=117;
Error de restricción CHECK
```

Consultando una vista

- **Usando WITH CHECK OPTION**
- Con la cláusula "with check option", no se permiten modificaciones en aquellos campos que afecten a los registros que retorna la vista. Es decir, no podemos modificar el campo “department_id" porque al hacerlo, tal registro ya no aparecería en la vista; si podemos actualizar los demás campos.
- La misma restricción surge al ejecutar un "insert" sobre la vista; solamente podemos agregar registros con el valor “30" para “department_id“.

Consultando una vista

- **Usando WITH READ ONLY**

```
CREATE  OR REPLACE VIEW empvu30
AS SELECT *
FROM employees
WHERE department_id=30
WITH READ ONLY;
```

```
DELETE FROM empvu30
WHERE employee_id=117;

DELETE FROM empvu30
ERROR at line 1:
ORA-01752:Cannot delete from view without exactly
one key-preserved table
```

Índices

- Objeto usado por Oracle para **aumentar** la **velocidad** de la **recuperación** de **filas** usando un puntero.
- Reduce operaciones de entrada y salida usando un método de rápido acceso que permite localizar los datos de forma rápida, siendo independiente de la tabla que apunta, con lo que pueden ser creados o borrados en cualquier momento sin que afecte a las tablas a las que apunta.
- Es automáticamente usado y mantenido por Oracle.

Índices

- **PRIMARY KEY** o **UNIQUE KEY**→Creación automática
- Manualmente: Los usuarios pueden crear índices no únicos en columnas para acelerar el tiempo de acceso a las filas.

```
CREATE INDEX nombre_indice
ON TABLE (column, [column….);
```

Índices

- **Ejemplo**: Aumenta la velocidad de acceso de consulta en la columna “first_name” de la tabla “Employees”.

```
CREATE INDEX emp_ename_idx
ON employees(first_name);
```

Índices

Conveniente	No Conveniente
Columna usada frecuentemente en WHERE	Si las columnas no se usan en WHERE
Columna contiene un amplio rango de valores	La tabla es actualizada frecuentemente
Tabla grande y la mayoría de las consultas esperadas devuelven menos de un 2-4% de las filas	Tabla pequeña y mayoría de las consultas esperadas devuelven más de un 2-4% de las filas
Dos o más columnas se usan mucho para unir dos tablas	
Contiene un elevado número de valores nulos	

Índices

- Consulta de índices

```
SELECT index_name, column_name
FROM user_ind_columns
WHERE table_name='EMPLEADOS';
```

- Borrado de índices

```
DROP INDEX nombre_índice;
```

Esquemas de Usuario en Oracle

- **Oracle**

- En Oracle se denomina **esquema** al **conjunto** de **objetos** que es propiedad de un usuario concreto.

- Dichos objetos pueden ser tablas, vistas, índices, procedimientos, funciones, triggers, etc…

- Es un planteamiento totalmente distinto al de MySQL, donde un **esquema** es, a grandes rasgos, una **base de datos**.

Gestión de Usuarios

- **Creación de Usuarios**
- Es necesario el privilegio de sistema **CREATE USER**. Normalmente sólo lo tiene el usuario **administrador.**
- No es posible la conexión del usuario creado a menos que posea el privilegio de sistema ***CREATE SESSION.***

```
CREATE USER <usuario>
IDENTIFIED BY <contraseña>/EXTERNALLY
DEFAULT TABLESPACE <espacio>
TEMPORARY TABLESPACE <espacio>/<grupo_espacios>
QUOTA <xx>/UNLIMITED ON <espacio>
PROFILE <perfil>
PASSWORD EXPIRE
ACCOUNT LOCK/UNLOCK;
```

Gestión de Usuarios

- **Creación de Usuarios**
- **EXTERNALLY**: El usuario se autentifica por sistema operativo
- **DEFAULT TABLESPACE**: Indica aquel espacio de almacenamiento donde se crearán los objetos del esquema del usuario cuando al hacerlo no se indica ninguno en particular.
- **TEMPORARY TABLESPACE**: Indica el espacio o grupo de espacios de almacenamiento para los segmentos temporales requeridos por el usuario.
- **QUOTA**: Indica la cantidad máxima de espacio que un usuario puede utilizar en un determinado espacio de almacenamiento. Puede indicarse cuota en múltiples espacios al mismo tiempo. Por defecto no se tiene cuota en ningún espacio de almacenamiento. Indicando *UNLIMITED,* es ilimitado el espacio a usar

Gestión de Usuarios

- **Creación de Usuarios**
- **PROFILE**: Indica el perfil a asignar al usuario (especifica limitaciones en recursos del sistema y restricciones). Si se omite se asigna el perfil *DEFAULT.*
- **PASSWORD EXPIRE**: Fuerza al usuario a cambiar la clave antes de conectarse a la base de datos.
- **ACCOUNT LOCK**: Bloquea la cuenta de usuario deshabilitando el acceso
- **ACCOUNT UNLOCK**: Desbloquea la cuenta de usuario y permite el acceso.

Gestión de Usuarios

- Ejemplo

```
SQL> CREATE USER ejemplo
  2  IDENTIFIED BY ejemplo
  3  DEFAULT TABLESPACE MARKETING
  4  QUOTA 2M ON MARKETING
  5  PROFILE COMERCIAL
  6  PASSWORD EXPIRE
  7  ACCOUNT LOCK;
```

Gestión de Usuarios

- **Modificación de usuarios**
- Los usuarios pueden cambiar sus propias claves, sin embargo, para cambiar cualquier otro parámetro es necesario el privilegio “ALTER USER”.

```
ALTER USER <usuario>
IDENTIFIED BY <contraseña>/EXTERNALLY
DEFAULT TABLESPACE <espacio>
TEMPORARY TABLESPACE <espacio>/<grupo_espacios>
QUOTA <xx>/UNLIMITED ON <espacio>
DEFAULT ROLE <role>/ALL/ALL EXCEPT <role>/NONE
PROFILE <perfil>
PASSWORD EXPIRE
ACCOUNT LOCK/UNLOCK;
```

Gestión de Usuarios

- Ejemplo

```
SQL> ALTER USER ejemplo
  2  ACCOUNT UNLOCK;
```

Gestión de Usuarios

- **Borrado de usuarios**
- Al borrar un usuario, en el esquema asociado todos sus **objetos desaparecen.**
- Una posible solución para que permanezca el usuario y los objetos pero impedir la conexión es **revocar** el privilegio "**CREATE SESSION**"
- No es posible eliminar un usuario que permanezca conectado a la base de datos.
- Es necesario tener el privilegio de sistema "**DROP USER**".
- Oracle no borra esquemas de usuario no vacíos a menos que se indique **CASCADE.**

```
DROP USER <usuario> CASCADE;
```

Gestión de Perfiles

- **CREACIÓN DE PERFILES**
- Está constituido por un conjunto de **límites** de **recursos** de la base de datos. Diferentes perfiles pueden ser asignados a diferentes usuarios. Es necesario el privilegio de sistema "**CREATE PROFILE**"
- Existe un perfil por defecto o ***DEFAULT***. Inicialmente todos los recursos en él tienen valor ***UNLIMITED,*** por lo que es conveniente modificarlo.
- Un usuario al que no se le asigna perfil posee el perfil ***DEFAULT.***
- Aquellos recursos para los que en el perfil asignado no se ha definido un valor, o se ha indicado *DEFAULT*, toman el valor que tiene el perfil por defecto.

```
CREATE PROFILE <nombre_perfil>
 LIMIT <parámetros> <valor>/UNLIMITED/DEFAULT;
```

Gestión de Perfiles

Parámetros de Recursos	Descripción
Sessions_per_user	Número de sesiones concurrentes
Connect_time	Tiempo total para una sesión (minutos)
Idle_time	Tiempo de inactividad continua en una sesión (minutos)

Gestión de Perfiles

Parámetros de contraseñas	Descripción
Failed_login_attempts	Número de intentos fallidos de conexión antes del bloqueo
Password_life_time	Número de días en que la clave es válida para autentificación
Password_grace_time	Periodo de gracia donde se permite la conexión pero se notifica la necesidad de cambiar la clave
Password_reuse_time	Número de días en los cuales la contraseña no puede reutilizarse
Password_reuse_max	Número de cambios de clave necesarios antes de poder reutilizar la clave actual
Password_lock_time	Número de días que la cuenta es bloqueada después de un cierto número de fallos de conexión

Gestión de Perfiles

- **Ejemplo**

```
SQL> CREATE PROFILE Comercial LIMIT
  2  FAILED_LOGIN_ATTEMPTS 5
  3  PASSWORD_LIFE_TIME 60
  4  PASSWORD_REUSE_TIME 60
  5  PASSWORD_REUSE_MAX 5;
```

Gestión de Perfiles

- **MODIFICACIÓN DE PERFILES**
- Es necesario poseer el privilegio de sistema "**ALTER PROFILE**"
- Los valores modificados no afectan a las sesiones en curso.

```
ALTER PROFILE <perfil>
 LIMIT <parámetros> <valor>/UNLIMITED/DEFAULT;
```

Gestión de Perfiles

```
SQL> ALTER PROFILE Comercial LIMIT
  2  FAILED_LOGIN_ATTEMPTS 3;
```

Gestión de Perfiles

- **ASIGNACIÓN DE PERFILES**
- Los perfiles no pueden asignarse a roles ni a otros perfiles, solo a usuarios.
- Se puede realizar durante la creación del usuario (**CREATE USER**) o posteriormente (**ALTER USER**).
- Un usuario solo puede tener un perfil asignado a la vez.
- Las asignaciones de perfiles no afectan a las sesiones activas.

```
SQL> CREATE USER prueba IDENTIFIED BY prueba
  2  PROFILE Comercial;
```

Gestión de Perfiles

- **BORRADO DE PERFILES**
- Debe poseerse el privilegio de sistema **DROP PROFILE**. El perfil DEFAULT no puede borrarse.
- Para eliminar un perfil asignado a un usuario debe usarse la opción **CASCADE**. Si se borra un perfil asociado a un usuario, a este se le asigna de forma automática el perfil DEFAULT.
- El borrado de un perfil no afecta a las sesiones en curso.

```
DROP PROFILE <perfil>;

DROP PROFILE <perfil> CASCADE;
```

Gestión de Privilegios

- Derecho a ejecutar un tipo determinado de sentencia SQL o a acceder a un objeto de otro usuario. Pueden asignarse a usuarios o, preferiblemente, a roles. Es importante no excederse en la concesión de privilegios.
- ***De sistema**:* Permite realizar determinadas acciones en la base de datos (Por ejemplo, crear espacios de almacenamiento, crear usuarios, …) o en cualquier esquema
- ***Sobre objetos**:* Permite a un usuario acceder y manipular o ejecutar objetos concretos (tablas, vistas, secuencias, procedimientos, funciones o paquetes). El permiso puede concederse con la posibilidad de que el usuario se lo de a otros.

Gestión de Privilegios

Objeto	Privilegio de Sistema
DATABASE	ALTER DATABASE, ALTER SYSTEM, AUDIT SYSTEM
TABLAS-ÍNDICES	CREATE TABLE, CREATE ANY TABLE / CREATE ANY INDEX, ALTER ANY TABLE / ALTER ANY INDEX, DROP ANY TABLE / DROP ANY INDEX, DELETE ANY TABLE / DROP ANY TABLE / INSERT ANY TABLE / UPDATE ANY TABLES / SELECT ANY TABLE
PROCEDURES	CREATE ANY PROCEDURE, ALTER ANY PROCEDURE, DROP ANY PROCEDURE, EXECUTE ANY PROCEDURE
PROFILES	CREATE PROFILE, ALTER PROFILE, DROP PROFILE
ROLES	CREATE ROLE, ALTER ANY ROLE, DROP ANY ROLE, GRANT ANY ROLE
USUARIOS	CREATE USER, ALTER USER, DROP USER

La cláusula ANY en cualquier privilegio indica que los usuarios a los que se les conceda tienen dicho privilegio en cualquier esquema.

Gestión de Privilegios

- **Privilegios de sistema**
- Para que un usuario pueda otorgar un privilegio de sistema, bien debe haberse otorgado con **ADMIN OPTION** (que permite a aquel a quien se le concede el privilegio poder otorgarlo) o haber sido concedido el privilegio **GRANT ANY PRIVILEGE**

Gestión de Privilegios

- **Privilegios de sistema**

```
SQL> GRANT CREATE ANY TABLE TO Prueba;

Grant succeeded.

SQL> GRANT CREATE SESSION TO Prueba;

Grant succeeded.
```

Gestión de Privilegios

- **Privilegios de sistema**
- Cualquier usuario con la opción ADMIN OPTION sobre un privilegio puede revocarlo. Quien lo hace no tiene porque ser el usuario que originalmente lo otorgó.

```
SQL> REVOKE CREATE ANY TABLE FROM Prueba;

Revoke succeeded.
```

Gestión de Privilegios

Objeto	Privilegios sobre objetos
TABLAS	ALTER, INDEX, SELECT, DELETE, INSERT, UPDATE, FLASHBACK, REFERENCES
VISTAS	DELETE, REFERENCES, FLASHBACK, SELECT, INSERT, UPDATE
SECUENCIAS	ALTER, SELECT
FUNCIONES, PAQUETES, PROCEDIMIENTOS	EXECUTE

Gestión de Privilegios

- **Privilegios sobre objetos**
- Con **ALL PRIVILEGES** se otorgan todos los privilegios sobre el objeto
- Con **PUBLIC** otorga el privilegio a todos los usuarios.
- La cláusula **GRANT OPTION** permite a aquel a quien se le concede el privilegio poder otorgarlo.

```
GRANT <privilegio>/ALL PRIVILEGES <columna>/ON <esquema>.objeto
    TO <usuario>/<rol>/PUBLIC;

GRANT <privilegio>/ALL PRIVILEGES ON <esquema>.objeto
    TO <usuario>/<rol>/PUBLIC WITH GRANT OPTION;
```

Gestión de Privilegios

- **Privilegios sobre objetos**

```
SQL> GRANT SELECT, INSERT, UPDATE ON HR.EMPLOYEES TO Prueba;

Grant succeeded.
```

Gestión de Privilegios

- **Privilegios sobre objetos**
- Quien otorga privilegios solo puede revocarlos a aquellos usuarios a quienes se los ha concedido.

```
SQL> REVOKE INSERT ON HR.EMPLOYEES FROM Prueba;

Revoke succeeded.
```

Gestión de Roles

- Es un **grupo de privilegios**, de sistema o sobre objetos, a los que se les da un nombre y pueden ser asignados a otros usuarios y roles.
- Pueden otorgarse a **cualquier usuario o rol**, pero no a sí mismo.
- Su **nombre** es **único** en la base de datos, distinto a cualquier otro nombre de usuario o rol.
- No pertenecen a **ningún esquema**.

Gestión de Roles

Roles Predefinidos	
CONNECT	**RESOURCE**
DBA	EXP_FULL_DATABASE
IMP_FULL_DATABASE	DELETE_CATALOG_ROLE
EXECUTE_CATALOG_ROLE	SELECT_CATALOG_ROLE

- Se recomienda crear roles específicos en cada base de datos y asignarles los permisos necesarios, evitando el uso de roles predefinidos, con lo que no surgirán problemas si éstos quedan obsoletos en futuras versiones

Gestión de Roles

- **Creación de roles**
- Debe poseerse el privilegio CREATE ROLE.
- CREATE ROLE <nombrerol>;

```
SQL> CREATE ROLE Permisos;

Role created.
```

Gestión de Roles

- **Asignar roles a usuarios**
- Para que un usuario pueda otorgar un rol debe habérsele concedido con **ADMIN OPTION**, poseer el privilegio **GRANT ANY ROLE**, o haberlo creado.

```
GRANT <rol> TO <usuario>/<rol>/PUBLIC;

GRANT <rol> TO <usuario>/<rol>/PUBLIC WITH ADMIN
OPTION;
```

```
SQL> GRANT CREATE SESSION, CREATE TABLE TO Permisos;

Grant succeeded.

SQL> CREATE USER EjRol IDENTIFIED BY EjRol;

User created.

SQL> GRANT Permisos TO EjRol;
```

Gestión de Roles

- **Desasignación de roles**

- Puede hacerlo cualquier usuario con la opción ADMIN OPTION para un rol, también aquellos usuarios con el privilegio GRANT ANY ROLE.

```
    GRANT <rol> TO <usuario>/<rol>/PU

    GRANT <rol> TO <usuario>/<rol>/PU
OPTION;
```

```
SQL> REVOKE Permisos FROM EjRol;
```

Gestión de Roles

- **Borrado de roles**

- Debe poseerse el privilegio DROP ANY ROLE o haber sido concedido el rol con ADMIN OPTION
- DROP ROLE <rol>;

```
SQL> DROP ROLE Permisos;

Role dropped.
```

Ruegos y Preguntas